

Transactions

Shahram Ghandeharizadeh

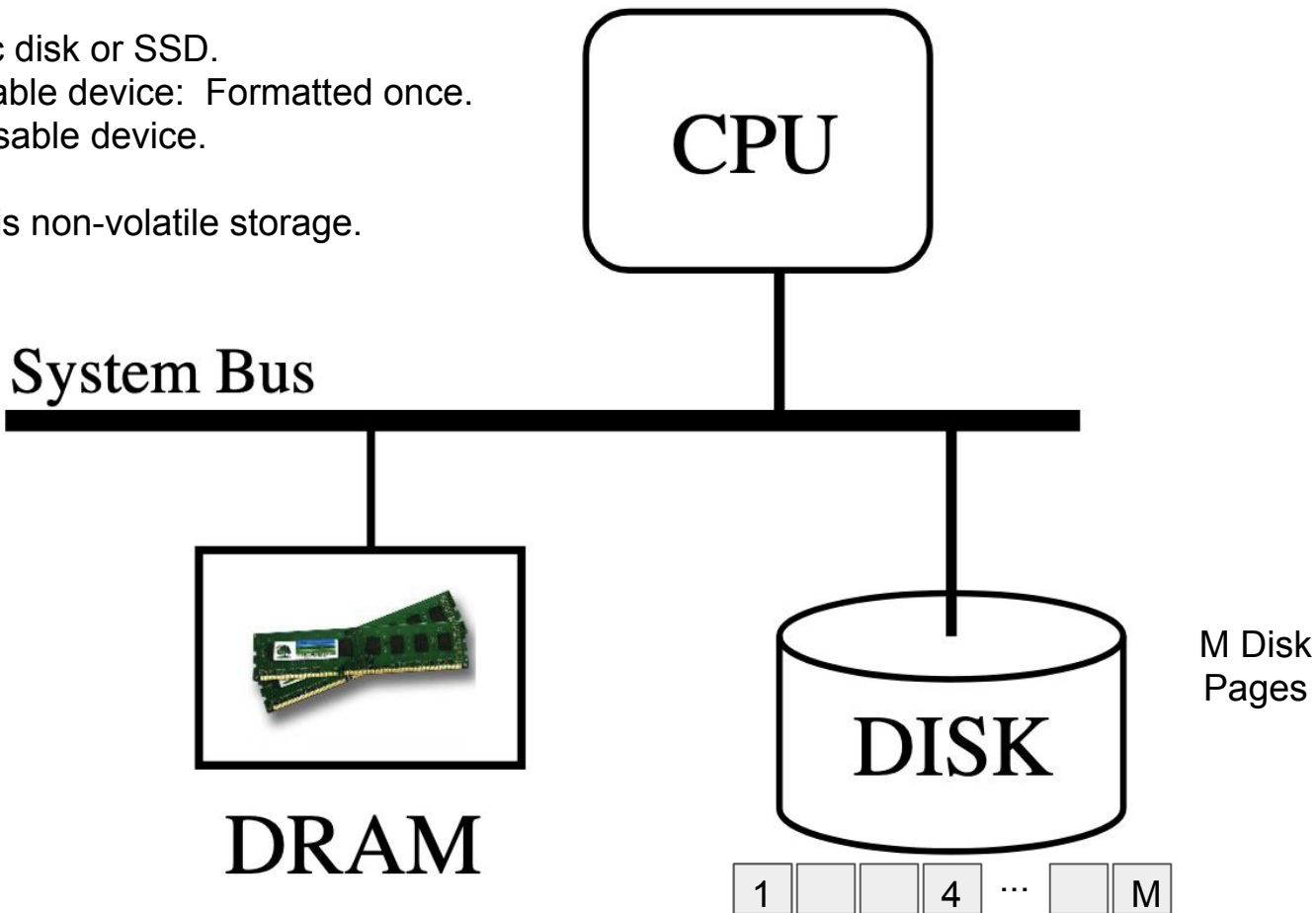
Hardware Organization

Disk may be a magnetic disk or SSD.

Disk is a block addressable device: Formatted once.

DRAM is a byte-addressable device.

DRAM is volatile. Disk is non-volatile storage.

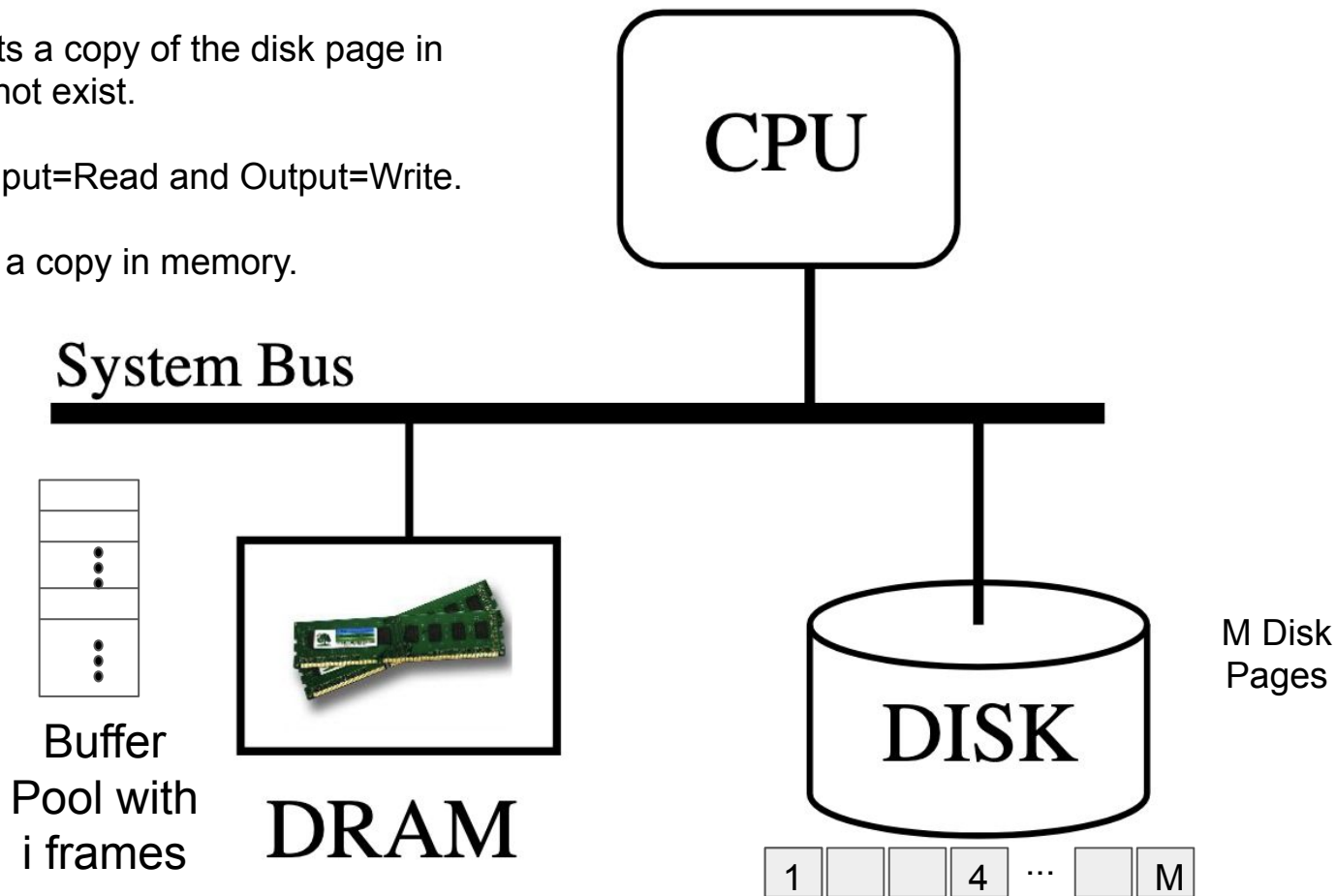


Disk Input/Output (I/O)

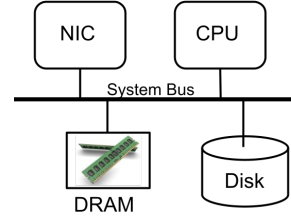
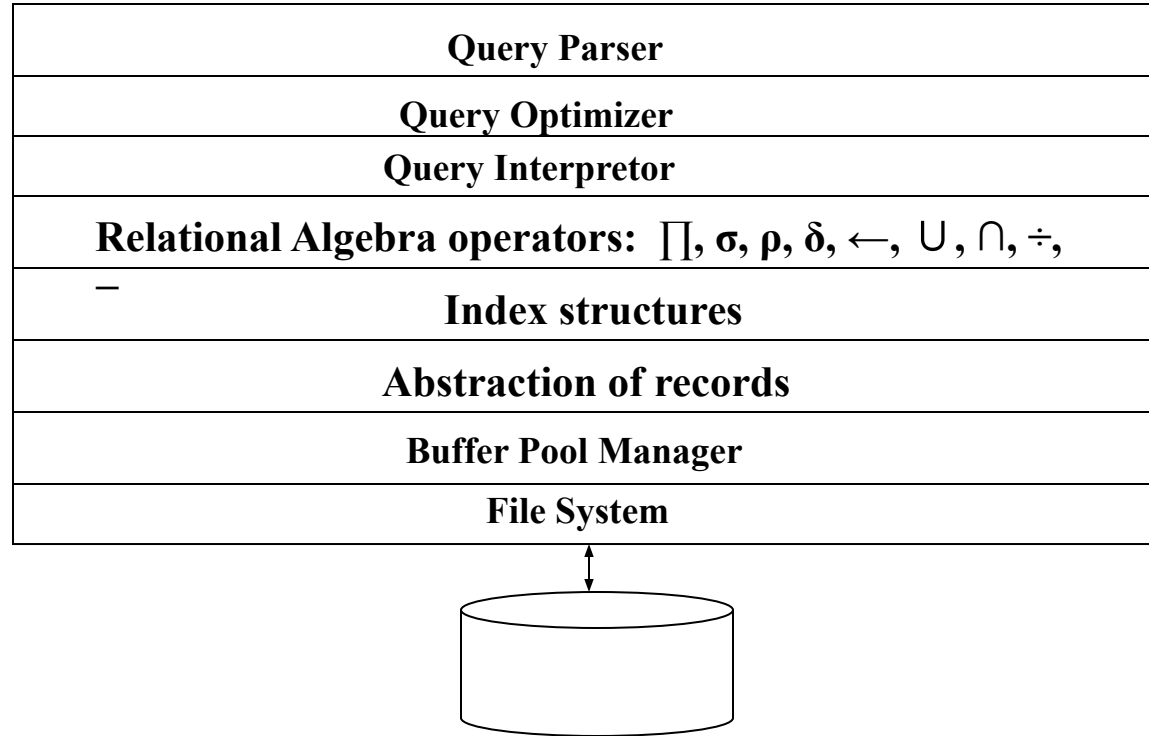
A Disk read constructs a copy of the disk page in memory if one does not exist.

Disk I/O stands for Input=Read and Output=Write.

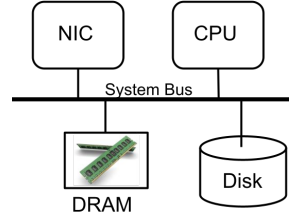
Disk write may leave a copy in memory.



Software Architecture of a DBMS



Implementation of σ



- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Shideh	4	4000	1

- $\sigma_{\text{Salary}=30,000}(\text{Employee})$

SS#	Name	Age	Salary	dno
4	Kathy	30	30000	5

- Process the select operator using a file scan (linear scan)

F1 = Open the file corresponding to Employee

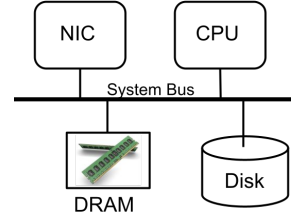
P = read first page of F1

While P is not null

For each record in P, if the record satisfies the selection predicate then produce as output

P = read next page of F1 /* P becomes null when EoF is reached */

Implementation of σ



- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Shideh	4	4000	1

SS#	Name	Age	Salary	dno
4	Kathy	30	30000	5

RID: (Page#, Slot #)

- $\sigma_{\text{Salary}=30,000}(\text{Employee})$

- Process the select operator using a file scan (linear scan)

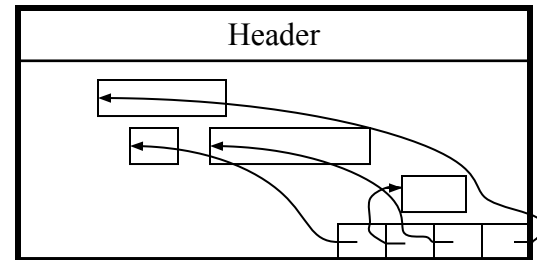
F1 = Open the file corresponding to Employee

P = read first page of F1

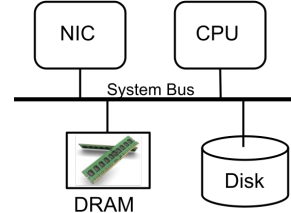
While P is not null

For each record in P, if the record satisfies the selection predicate then produce as output

P = read next page of F1



Implementation of σ



- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Shideh	4	4000	1

- $\sigma_{\text{Salary}=30,000}(\text{Employee})$

SS#	Name	Age	Salary	dno
4	Kathy	30	30000	5

Fetch the page from disk if not in the buffer pool

- Process the select operator using a file scan (linear scan)

F1 = Open the file corresponding to Employee

P = read first page of F1

While P is not null

For each record in P, if the record satisfies the selection predicate then produce as output

P = read next page of F1

Transaction Processing

- Database management systems (DBMSs) have historically been used in banking applications, e.g., transfer some funds from one account to another. The DBMS must ensure the consistency of the database by supporting logical operations that are a collection of instructions
- A **transaction** is a collection of operations that performs a single logical function. Each transaction is a unit of atomicity. Example: transfer 50 from John's account to Jane's
 - read(A)
 - $A = A - 50$
 - write(A)
 - read(B)
 - $B = B + 50$
 - write(B)



Transaction: Sample Java Code

```
try (Connection conn = DriverManager.getConnection(jdbcUrl, username, password);) {  
  
    // STEP 1 - Disable auto commit mode  
    conn.setAutoCommit(false);  
  
    try (PreparedStatement insertStmt = conn.prepareStatement(INSERT_USERS_SQL); PreparedStatement  
  
        // Create insert statement  
        insertStmt.setInt(1, 200);  
        insertStmt.setString(2, "Tony");  
        insertStmt.setString(3, "tony123@gmail.com");  
        insertStmt.setString(4, "US");  
        insertStmt.setString(5, "secret");  
        insertStmt.executeUpdate();  
  
        // Create update statement  
        updateStmt.setString(1, "Ram");  
        updateStmt.setInt(2, 200);  
        updateStmt.executeUpdate();  
  
        // STEP 2 - Commit insert and update statement  
        conn.commit();  
        System.out.println("Transaction is committed successfully.");  
    } catch (SQLException e) {  
        printSQLException(e);  
        if (conn != null) {  
            try {  
                // STEP 3 - Roll back transaction  
                System.out.println("Transaction is being rolled back.");  
                conn.rollback();  
            } catch (Exception ex) {  
                ex.printStackTrace();  
            }  
        }  
    }  
}
```

Transactions

- Application developer is responsible for:
 - Correctness of the program that is designated as a transaction.
- DBMS is responsible for:
 - **Atomicity.** Either all operations of the transaction are properly reflected in the database or none are.
 - **Consistency.** Execution of a transaction in isolation preserves the consistency of the database.
 - **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions.
 - **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

Failures

- Application, where the application corrupts the database.
- Bohr bugs: DBMS bugs that can be recreated.
- Heisenbugs: DBMS bugs that cannot be recreated.
- Denial of service attacks.
- Lost network packets.
- Hardware failures.
 - Disk.
- Power failures.
- Run-time errors, e.g., deadlocks.

Failures

- Application, where the application corrupts the database.
- Bohr bugs: DBMS bugs that can be recreated.
- Heisenbugs: DBMS bugs that cannot be recreated.
- Denial of service attacks.
- Lost network packets.
- Hardware failures.
 - Disk.
- Power failures.
- Internal failures, e.g., deadlocks.

Data Backup



Conceptually

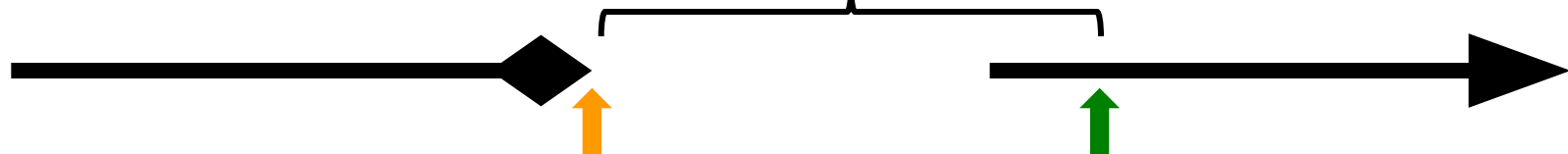
- Some failures (deadlocks, disk failures) may not disrupt service; only degrade system performance.



Detect Disk Failure

- Other failures may disrupt service.

Recovery Phase



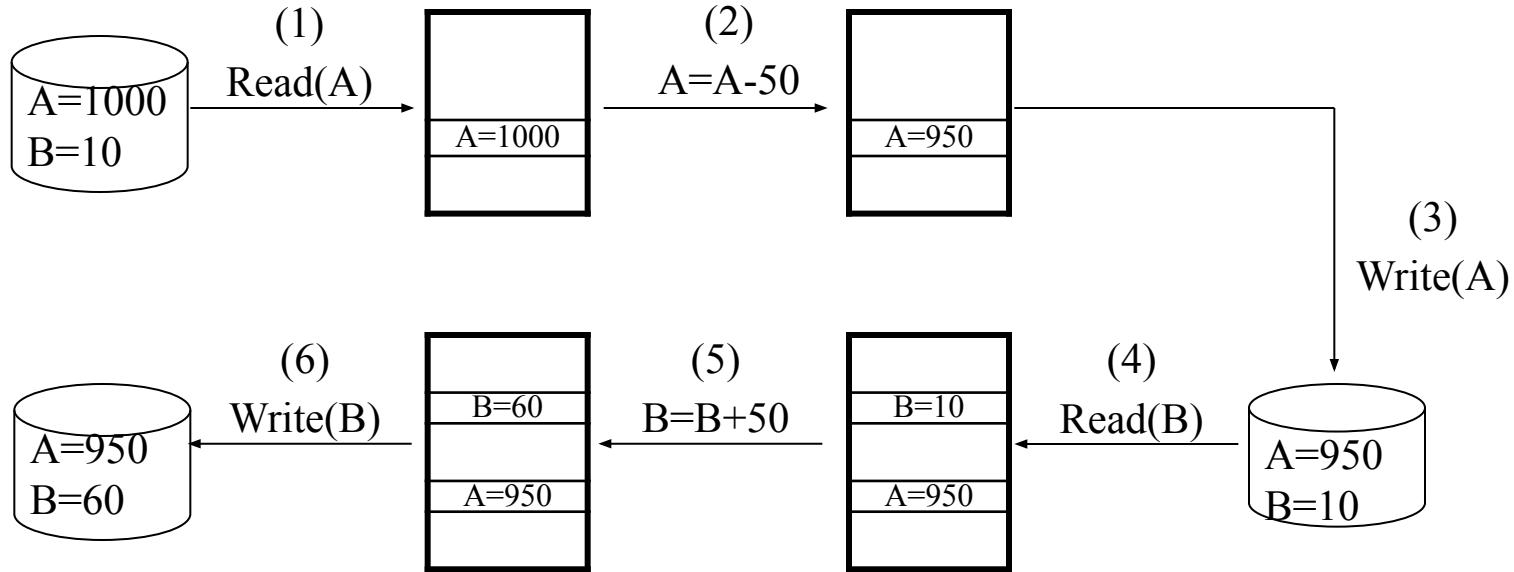
Detect Corrupt
database

Resume Xact
processing

Example Transaction

- A **transaction** is a collection of operations that performs a single logical function. Each transaction is a unit of atomicity. Example: transfer 50 from John's account to Jane's
 - read(A)
 - A=A-50
 - write(A)
 - read(B)
 - B=B+50
 - write(B)

SYSTEM FAILURE EXAMPLE



- Assume a power failure at Step 4. When the system recovers the database state is inconsistent because: 1) $A+B \neq 1010$, and 2) logically B has lost 50.

Example of Fund Transfer

- Transaction to transfer \$50 from account A to account B:
 1. **read**(A)
 2. $A := A - 50$
 3. **write**(A)
 4. **read**(B)
 5. $B := B + 50$
 6. **write**(B)
- **Atomicity requirement**
- **Consistency requirement**
- **Isolation requirement**
- **Durability requirement**

LOG-BASED RECOVERY

- Two approach to ensure atomicity of a transaction: Log-based, shadow paging.
- *Log-based recovery*: The system maintains a database log. Each log record may contain the following possible values:
 1. transaction name: each transaction must have a unique identifier
 2. data item name: unique name of data item that is written
 3. old value: the value of data item prior to the write operation
 4. new value: the value that data item will have after the write operation
- The various log records are:
 - $\langle T_i, start \rangle$: Transaction T_i has started
 - $\langle T_i, X, V_1, V_2 \rangle$: Transaction T_i has written data item X . The original value of X was V_1 , its new value is V_2 .
 - $\langle T_i, commit \rangle$: Transaction T_i has committed

LOG-BASED RECOVERY (Cont...)

- Before a transaction modifies the actual data page, it must generate a log record for its proposed change and write this record to the log file.

– T_1 Log records

$\langle T_1, \text{start} \rangle$

read(A) $\langle T_1, A, 1000, 950 \rangle$

A=A-50

write(A)

read(B) $\langle T_1, B, 10, 60 \rangle$

B=B+50

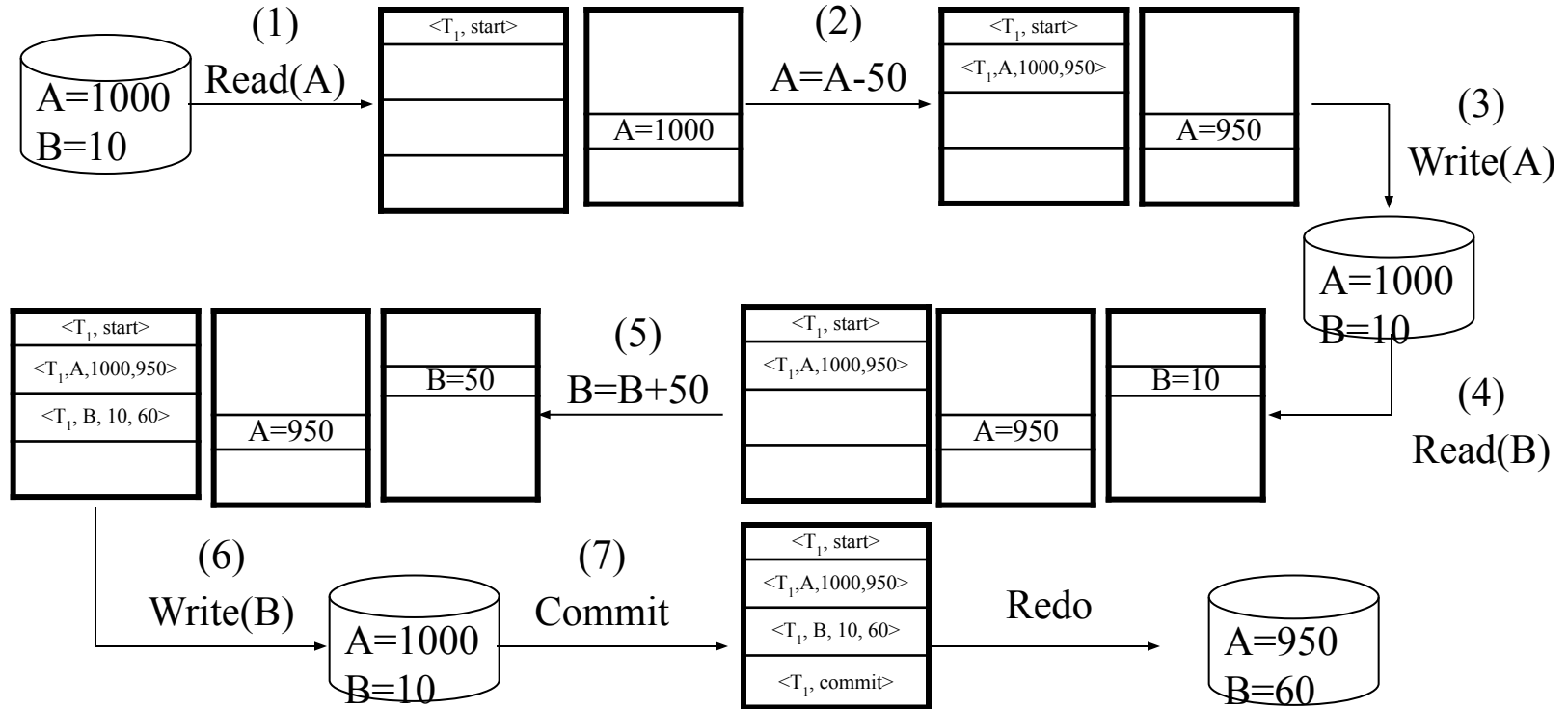
write(B)

$\langle T_1, \text{commit} \rangle$

LOG-BASED RECOVERY (Cont...)

- There are two alternative approaches to logging: deferred database modification, and immediate database modification.
- *Logging with deferred database modification*
 1. Record all database modifications performed by a transaction T_i in a log file, no physical updates are performed on the data items.
 2. When T_i commits:
 - a) generate a commit log record for T_i to the log file
 - b) write the log file to the disk drive
 - c) execute the updates proposed in the log file using redo(T_i)
 - redo(T_i) sets the value of all data item updates by transaction T_i to their new value.
 - In the presence of failures, the system invokes redo(T_i) for each transaction that has a commit log record in the log file.
 - Redo is **idempotent**, repeating it more than one time has no side effect.

DEFERRED LOG-BASED RECOVERY (Cont...)



Deferred Database Modification (Cont.)

- During recovery after a crash, a transaction needs to be redone if and only if both $\langle T_i \text{ start} \rangle$ and $\langle T_i \text{ commit} \rangle$ are there in the log.
- Redoing a transaction T_i (**redo** T_i) sets the value of all data items updated by the transaction to the new values.
- Crashes can occur while
 - the transaction is executing the original updates, or
 - while recovery action is being taken
- example transactions T_0 and T_1 (T_0 executes before T_1):
- Default values: A=1000 B=2000 C=700

T_0 : **read** (A)

A: - A - 50

Write (A)

read (B)

B:- B + 50

write (B)

T_1 : **read** (C)

C:- C- 100

write (C)

Deferred Database Modification (Cont.)

- Below we show the log as it appears at three instances of time.

$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$
$\langle T_0, A, 950 \rangle$	$\langle T_0, A, 950 \rangle$	$\langle T_0, A, 950 \rangle$
$\langle T_0, B, 2050 \rangle$	$\langle T_0, B, 2050 \rangle$	$\langle T_0, B, 2050 \rangle$
	$\langle T_0 \text{ commit} \rangle$	$\langle T_0 \text{ commit} \rangle$
	$\langle T_1 \text{ start} \rangle$	$\langle T_1 \text{ start} \rangle$
	$\langle T_1, C, 600 \rangle$	$\langle T_1, C, 600 \rangle$
		$\langle T_1 \text{ commit} \rangle$
(a)	(b)	(c)

- If log on stable storage at time of crash is as in case:
 - (a) No redo actions need to be taken
 - (b) redo(T_0) must be performed since $\langle T_0 \text{ commit} \rangle$ is present
 - (c) **redo**(T_0) must be performed followed by redo(T_1) since $\langle T_0 \text{ commit} \rangle$ and $\langle T_1 \text{ commit} \rangle$ are present

LOG-BASED RECOVERY (Cont...)

- *Logging with immediate database modification*
 1. All the write operations on the data items are performed immediately
 2. A transaction generates log records as it updates records
 3. The buffer pool maintains the dependency between the log records and the buffer pool frames containing the updated data item
 - (a) Before a buffer pool frame is written to the disk, its corresponding log record must be written to the disk.
 - (b) A transaction does not commit until its commit log record is written to the disk.
- In the presence of failures, with immediate modification, the system must analyze the log record and perform Redo/Undo operations.
- With deferred database modification, the system eliminates Undo operations during the recovery period. However, when processing a transaction, the system must still perform Redo operations.

Immediate Database Modification Example

Log	Write	Output
$\langle T_0 \text{ start} \rangle$		
$\langle T_0, A, 1000, 950 \rangle$		
$\langle T_0, B, 2000, 2050 \rangle$		
	$A = 950$	
	$B = 2050$	
$\langle T_0 \text{ commit} \rangle$		
$\langle T_1 \text{ start} \rangle$		
$\langle T_1, C, 700, 600 \rangle$		
	$C = 600$	
		B_B, B_C
$\langle T_1 \text{ commit} \rangle$		
		B_A
Note: B_X denotes block containing X.		

Immediate DB Modification Recovery Example

Below we show the log as it appears at three instances of time.

$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$
$\langle T_0, A, 1000, 950 \rangle$	$\langle T_0, A, 1000, 950 \rangle$	$\langle T_0, A, 1000, 950 \rangle$
$\langle T_0, B, 2000, 2050 \rangle$	$\langle T_0, B, 2000, 2050 \rangle$	$\langle T_0, B, 2000, 2050 \rangle$
	$\langle T_0 \text{ commit} \rangle$	$\langle T_0 \text{ commit} \rangle$
	$\langle T_1 \text{ start} \rangle$	$\langle T_1 \text{ start} \rangle$
	$\langle T_1, C, 700, 600 \rangle$	$\langle T_1, C, 700, 600 \rangle$
		$\langle T_1 \text{ commit} \rangle$
(a)	(b)	(c)

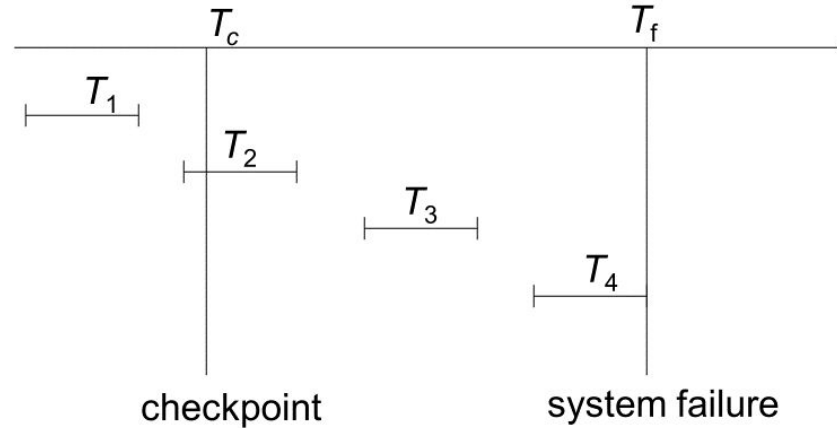
Recovery actions in each case above are:

- (a) undo (T_0): B is restored to 2000 and A to 1000.
- (b) undo (T_1) and redo (T_0): C is restored to 700, and then A and B are set to 950 and 2050 respectively.
- (c) redo (T_0) and redo (T_1): A and B are set to 950 and 2050 respectively. Then C is set to 600

CHECKPOINTING

- Motivation: In the presence of failures, the system consults with the log file to determine which transaction should be redone and which should be undone. There are two major difficulties:
 - 1) the search process is time consuming
 - 2) most transactions are okay as their updates have made it to the database (the system performs wasteful work by searching through and redoing these transactions).
- Approach: perform a checkpoint that requires the following operations:
 1. output all log records from main memory to the disk
 2. output all modified (dirty) pages in the buffer pool to the disk
 3. output a log record *<checkpoint>* onto the log file on disk

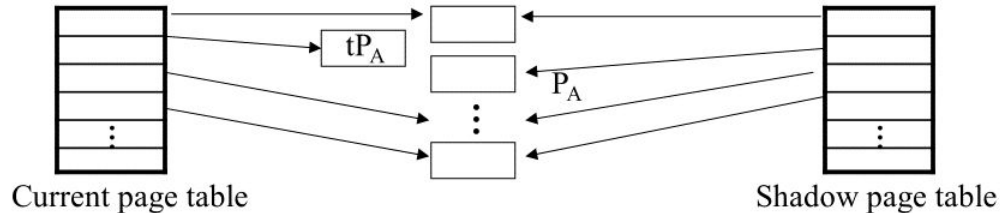
Operations Performed During Recovery



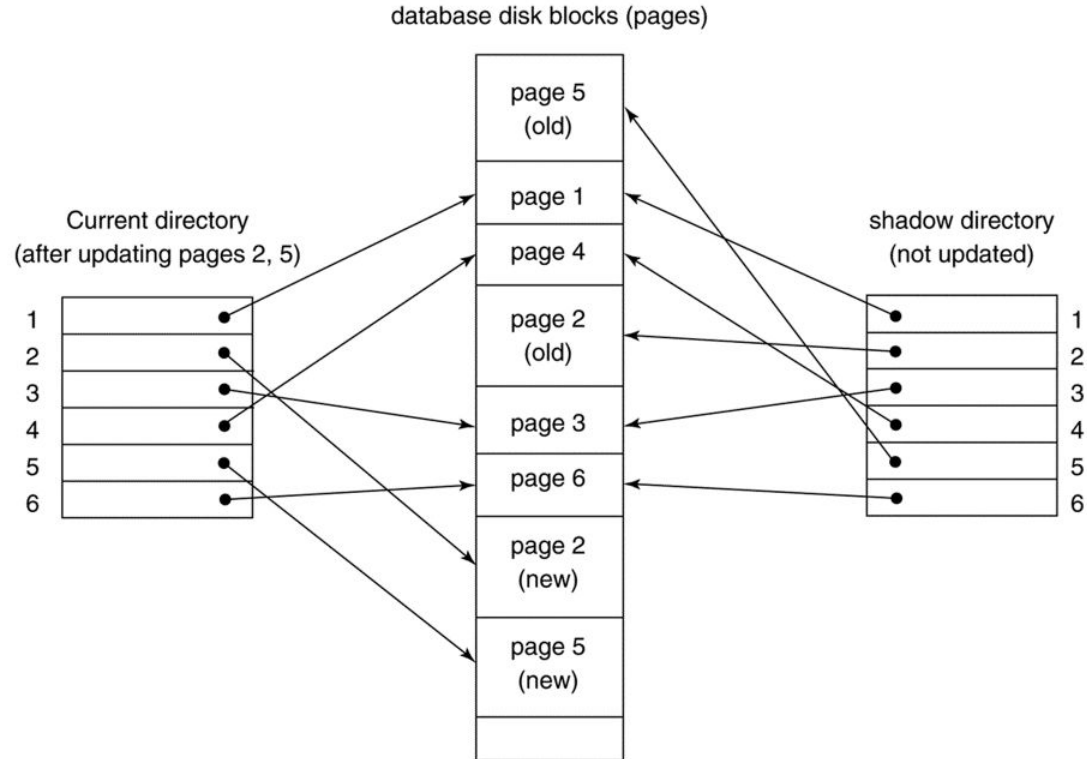
- T_1 can be ignored (updates already output to disk due to checkpoint)
- T_2 and T_3 redone.
- T_4 undone
- For each transaction T_k after and during the checkpoint, if $\langle T_k, \text{commit} \rangle$ log record appears in the log file then execute $\text{Redo}(T_k)$
- For each transaction T_k after the checkpoint, if no $\langle T_k, \text{commit} \rangle$ log record appears in the log file then execute $\text{Undo}(T_k)$

SHADOW PAGING

- *Shadow paging*: takes advantage of the structure of the file system. It generates no log records and is different than the log-based recovery techniques. Assume the file system is organized as a set of pages and a page table that keeps track of the pages in the file system.
- The system maintains two page tables during the life of a transaction. The current page table and the shadow page table. When transaction T_i starts, both page tables are identical. The shadow page table never changes. The current page table may change when a transaction performs a write operation. All input/output operations use the current page table to locate the database pages on a disk.



An example of shadow paging.



SHADOW PAGING (Cont...)

When transaction T_i issues a write(A) command, the write operation is executed as follows (assuming data item A resides on page P_A):

1. If page P_A is not already in main-memory then issue input(P_A)
2. If this is the first write operation on page P_A by transaction T_i then:
 - (a) allocate a new disk page (call it tP_A)
 - (b) copy P_A into tP_A
 - (c) modify the current page table so that the entry corresponding to P_A now points to tP_A
3. perform the update on the page pointed to by tP_A

SHADOW PAGING (Cont...)

- When transaction T_i commits:
 1. Ensure all buffer pages in memory that have been modified are flushed to the disk.
 2. output the current page table to disk. Do not overwrite the shadow page table because it might be needed for recovery from a crash.
 3. Output the disk address of the current page table to the fixed location in disk containing the address of the shadow page table. Thus, the current page table has become the shadow page table, and the transaction is committed.

Other details:

1. free the pages of the old shadow page table that are no longer necessary.
Requires reading the old shadow page table and free its pages. What happens when there is a crash?

SHADOW PAGING (Cont...)

Shadow paging suffers from following limitations:

1. data fragmentation: the pages of a file are dispersed across the surface of a disk
2. garbage collection: pages that are no longer accessible should be gathered as they are garbage now
3. What happens to auxiliary index structures?
4. Can multiple transactions update a file simultaneously? If so, what happens if two transactions try to update two different records that reside on the same disk page.